

All students should watch the videos and review the readings accompanying this LGA in preparation for this assignment and the next lecture.

Individual self-test questions

All students should answer this set of questions. You will compare your answers with your group during the next lecture period. Note, we may from time to time include trick questions.

- “FIFO” stands for First In, First Out and describes the Queue ADT.
- “LIFO” stands for Last In, First Out and describes the Stack ADT.
- The three principle operations on a stack are push, pop, and top. These operations allow interaction only with the top of the stack.
- The three principle operations on a queue are enqueue (or push), dequeue (or pop), and front. Which end of the queue does each operation affect?

Enqueue adds an object to the back of the queue, dequeue removes an item from the front, and front returns the element at the front of the queue.

Group questions

Distribute the following questions across the members of your group. You will share your solutions (and most importantly the *method* of your solutions) during the next lecture period. Divide up the questions so that each question has at least three solutions from different group members.

1. (Basic) Write a C++ function that, given a stack of integers, swaps the top two elements in the stack; if the stack contains fewer than two elements, the function should do nothing.

```
void swapTopTwo(stack<int> &s) {  
    if (s.size() < 2) return; // do nothing  
    int tempInt1 = s.top();    // remove and store first top integer  
    s.pop();  
    int tempInt2 = s.top();    // remove and store second top integer  
    s.pop();  
    s.push(tempInt1);          // push the former top element first  
    s.push(tempInt2);  
}
```

2. (Intermediate) You are given a large box. Inside the box is another, smaller box; inside that box is a yet smaller box; and so forth. There are n boxes in total. Each box is labeled with a unique label.

Your job is to separate out all the boxes. Another worker is waiting to take each box as soon as you open it and remove its contents; you only have time enough to quickly record the label on the box before it is taken away. You are only able to use a stack data structure for recording the labels. Your co-worker moves all of the boxes to a new location.

The next day, your job is to put all of the boxes back as they were - smaller into larger. However, your boss is a real stickler for efficiency, and will only let you place a smaller box into a larger box a total of $n - 1$ times - which means you cannot make any mistakes. Unfortunately, many of the boxes are similar in size, there are a lot of boxes, and your careless co-worker scattered the boxes around any which way. Good thing you have your stack!

Describe your algorithm for repacking all of the boxes.

We have to push each label to the stack as we open each box, because the box is taken away as soon as we remove the smaller boxes inside. This means the largest box label is at the *bottom* of the stack. Remove the top-most label from the stack - this is our starting box (i.e., the smallest). You can close and tape this box immediately. Remove the next label from the stack - this is the box surrounding the previous box, as originally it contained the smallest box, and was thus pushed to the stack just before it. Put the previous box into this box and seal the box. Repeat this process of getting the label from the top of the stack and packaging the corresponding box around the previous box. This will provide us with the correct repacking with the help of our LIFO stack!

3. (Intermediate) Same problem, but you are given a queue instead of a stack.

Assuming we push the labels to the queue in the same order as the stack (biggest to smallest), we must reverse our packaging order. Since we pushed the biggest box first, it will be the first label to pop from the queue. The next will be the box that goes inside that biggest box. Repeat until the queue is empty. Note we cannot tape up any of the boxes until they are all nested, which will make our job much harder!

4. (Intermediate) A palindrome is a string that reads the same forward and backward, such as “kayak”, “civic”, or “racecar”. Describe one way you can use a stack and a queue together to determine if a given string is a palindrome.

For each character in the string, push to both the stack and the queue. Now, loop through the length of the string, calling `top()` on the stack and `front()` on the queue and check if they are equal. Call `pop()` on both to move to the next character. If at any point during the loop the characters do not match, it *is not* a palindrome. If we get to the end of the loop, we know it *is* a palindrome. This works because the stack (LIFO) will return the word backwards while the queue (FIFO) will return the word forwards (read from left to right). By checking equivalence for each character, we determine whether each letter starting from the back and the front are equal, thus telling us whether or not the given string is a palindrome.

For example: kayak

stack will return: 'k' 'a' 'y' 'a' 'k'

queue will return: 'k' 'a' 'y' 'a' 'k'

is a palindrome

For example: palindrome

stack will return: 'e' 'm' 'o' 'r' 'd' 'n' 'i' 'l' 'a' 'p'

queue will return: 'p' 'a' 'l' 'i' 'n' 'd' 'r' 'o' 'm' 'e'

is not a palindrome

5. (Basic) What is the output of the following code? (Drawing the stack and queue may be helpful!)

```
stack<string> s;
queue<string> q;

s.push("green");
q.push("green");
s.push("grey");
q.push("grey");

s.pop();
q.pop();

cout << s.top() << endl;
cout << q.front() << endl;

s.push("yellow");
q.push("yellow");
s.push("purple");
q.push("purple");

s.pop();
q.pop();

cout << s.top() << endl;
cout << q.front() << endl;

s.push("orange");
q.push("orange");

s.pop();
q.pop();
s.pop();
q.pop();

cout << s.top() << endl;
cout << q.front() << endl;
```

```
green
grey
yellow
yellow
green
orange
```

For Further Exploration

Reverse Polish Notation (RPN) or postfix is an arithmetic expression notation where operators follow operands. For example: “3 + 4” is written as “3 4 +” in RPN. RPN notation may be used to write composite expressions such as “5 8 6 - ×”, which would be “5 × (8 - 6)” in infix notation.

Evaluation of expressions is accomplished using a stack. The expression is read left-to-right. As operands are encountered, they are pushed onto the stack. When an operator is encountered, the necessary number of operands are popped from the stack, the operator is applied, and the result is pushed back onto the stack.

Let’s trace through the example above, “5 8 6 - ×”: first, we push 5, then 8, and then 6 onto the stack. The first operator encountered is the minus operator; since 8 and 6 are the top two elements of the stack at that point, those operands are removed from the stack, and the difference (2) is pushed back onto the stack. This leaves the 5 and the 2 on the stack for the final multiplication operation.

One advantage of the RPN is that you never need parentheses to indicate the order of expression evaluation! HP’s line of pocket calculators are famous for their use of RPN and stack evaluation. Read more here: <https://www.hpmuseum.org/rpn.htm>.

Using the STL stack, can you write a simple program to evaluate arithmetic expressions in RPN?

```
#include <iostream>
#include <stack>
#include <string>

using namespace std;

// written assuming (for simplicity):
// - a properly formatted RPN
// - integer division
// - single digit integer inputs
int eval_RPN(string rpn) {
    stack<int> s;

    for (int i = 0; i < rpn.length(); i++) {
        int val1, val2;

        switch (rpn[i]) {
            case ' ': // skip spaces
                break;
            case '-': // subtract top two elements
                val2 = s.top();
                s.pop();
                val1 = s.top();
                s.pop();
                s.push(val1 - val2);
                break;
            case '+': // add top two elements
                val2 = s.top();
                s.pop();
                val1 = s.top();
                s.pop();
                s.push(val1 + val2);
                break;
            case '*': // multiply top two elements
                val2 = s.top();
                s.pop();
                val1 = s.top();
```

```
        s.pop();
        s.push(val1 * val2);
        break;
    case '/': // divide top two elements
        val2 = s.top();
        s.pop();
        val1 = s.top();
        s.pop();
        s.push(val1 / val2);
        break;
    default: // assume otherwise we have a number
        s.push(rpn[i] - 48); // ascii math
        break;
    }
}

return s.top(); // the last value left on the stack is our solution
}

int main() {
    cout << eval_RPN("3 4 +") << endl; // will print 7
    cout << eval_RPN("5 8 6 - *") << endl; // will print 10
}
```